

Security Assessment Report

Keystone Ecosystem — keyst.one, Commerce API, SDK
Acceptance Layer & Device Firmware

Analyst: **OxRobotnick**

Date: September 2026

Classification: **Private — Client Distribution Only**

1 systemic HIGH (CRITICAL-candidate) · 3 HIGH · 2 MEDIUM-HIGH · 4 MEDIUM · 4 LOW · 3
THEORETICAL

All CONFIRMED ratings backed by reproducible proof-of-concept executions

Security Assessment Report — Keystone Ecosystem (keyst.one & SDK/Firmware)

Client: Keystone (Keystone Hardware Wallet) **Analyst:** 0xRobotnick **Date:** September 2026

Classification: Private — client distribution only **Engagement:** External black-box assessment + open-source code review; active verification bounded and non-destructive

1. Executive Summary

This assessment examined the Keystone hardware-wallet ecosystem from four angles: the public web property (keyst.one), its commerce API surface (api.keyst.one, api-staging.keyst.one, shop.keyst.one), the vendor's public SDK repositories (the software bridge between host wallets and the hardware device), and the open-source device firmware.

The headline result is a systemic finding: the QR-signing acceptance channel between host software and the hardware wallet is unauthenticated end-to-end. Requests, results, and pairing payloads cross the camera boundary with no cryptographic binding and no mandatory session comparison in any SDK layer. Four independently executed proof-of-concept runs against the vendor's *published* packages demonstrate: acceptance of a substituted signing result belonging to a different session (LTC/BCH/DASH/DOGE flow — no binding check exists at all), acceptance of a signature response whose binding field is omitted (ETH/SOL/COSMOS/APTOS keyrings — check present but optional), client-side conversion of ordinary transactions into hash-only "blind signing" via a remotely-controlled configuration object, and valid signatures produced over silently truncated (empty) message data.

The device firmware itself was found to be well-engineered: every transaction parser reviewed fails closed, the firmware update chain enforces dual signature verification, and the entropy-generation path correctly chains three independent hardware RNG sources. The vulnerability class identified here lives entirely in the **host-side SDK**, between the device screen and the network broadcast — precisely the layer implicated in the industry's largest hardware-wallet losses (Bybit/Safe, \$1.5B, February 2025).

Severity summary: 1 systemic HIGH (CRITICAL-candidate in aggregate), 3 HIGH, 2 MEDIUM-HIGH, 4 MEDIUM, 4 LOW, 3 THEORETICAL. No inflated severities: every CONFIRMED rating is backed by a reproducible proof-of-concept; every unproven chain is labeled THEORETICAL.

2. Scope & Methodology

In scope: keyst.one (web), api.keyst.one, api-staging.keyst.one, shop.keyst.one (storefront), client JavaScript bundles, public KeystoneHQ repositories (keystone-sdk-web, keystone-sdk-base, keystone3-firmware, Smart-Contract-Metadata-Registry, Keystone-developer-hub), and the vendor's published npm packages (verified: @keystonehq/keystone-sdk 0.12.3, @keystonehq/base-eth-keyring 0.15.1 — dist code confirmed identical to repository logic).

Out of scope: device hardware attacks, side-channel analysis, user social engineering beyond documented flow abuse, any action creating real financial transactions.

Methodology (neutral description): 1. Passive infrastructure enumeration (DNS, certificate-transparency authorities, subdomain discovery, TLS configuration audit). 2. Unauthenticated web/API probing at conservative rates, with full respect for observed rate-limiting guardrails. 3. Complete client-bundle analysis: 103 production JavaScript bundles downloaded and mined for routes, parameters, configuration, and secrets — yielding 100% coverage of the API's client-known surface (13 routes). 4. Service enumeration of the storefront (1,306 routes crawled) with standard administrative-leak verification. 5. Manual source review of all five public repositories, prioritizing crypto-material handling, parser trust boundaries, and session binding. 6. Runtime proof-of-concept development against published packages and live endpoints, with each finding subjected to ≥ 3 verification variants before severity assignment (CONFIRMED / THEORETICAL discipline). 7. Cross-referencing against public threat intelligence and prior vendor audits.

3. Architecture Overview

- **Web edge:** keyst.one served via CDN with WAF (User-Agent filtering and behavioral rate rules; scanner-identifiable clients receive 403).
- **Web origin:** Express, Vite-built React SPA. Headers: HSTS (max-age 1y, no includeSubDomains/preload), X-Frame-Options SAMEORIGIN, nosniff, referrer-policy. **No Content-Security-Policy; no Permissions-Policy.**
- **Commerce API:** Express JSON API fronting a Shopify Storefront integration (13 routes fully mapped from client bundles; live verification on production and staging).
- **Storefront:** Shopify behind a separate Cloudflare edge; underlying store `cobo-vault.myshopify.com` (store id 2931753060, legacy brand). Native cart API live (`/cart*`), 17TRACK and SAP application proxies, public catalog JSON (15 products / 169 variants). Payment executes on Shopify-hosted checkout — card data does not traverse vendor infrastructure.
- **SDK layer:** `@keystonehq/keystone-sdk` (chain parsers, 14+ chains), `@keystonehq/base-eth-keyring` and siblings (wallet integrations), UR/CBOR QR protocol between host and device.
- **Firmware:** C + Rust hybrid; transaction parsers in Rust per chain; three secure elements; signed firmware update pipeline.
- **Third parties:** feature-flag service (cloud-hosted), CAPTCHA service (server-side verified), analytics (Mixpanel), error reporting (Sentry DSN in bundle), affiliate tracking, order tracking.

4. Findings — HIGH Severity (all runtime-verified against published packages)

4.1 [F15] SYSTEMIC — The QR acceptance channel is unauthenticated end-to-end

Severity: HIGH (systemic) — CRITICAL-candidate in aggregate **CWE:** CWE-345, CWE-354 **Status:** CONFIRMED (architectural property; three runtime PoCs demonstrate it per-path)

Finding. The SDK layer that accepts scanned results from the hardware device performs **no authentication or binding anywhere in the stack**. Verified across every layer:

Layer	Binding status
14+ chain parsers (<code>keystone-sdk-web/src/chains/*</code>)	Response <code>requestId</code> echoed verbatim; zero comparison operations exist in the codebase (verified by exhaustive source search — every match is an undefined-guard, never an equality check)
4 keyring packages (base-eth, sol, cosmos, aptos)	Comparison exists but is optional — skipped when the response omits the field (Finding 4.3)
Legacy-coin protocol (LTC/BCH/DASH/DOGE)	No comparison anywhere in the stack (Finding 4.2)
Signature-level validation	No path verifies a returned signature or rawTx against the transaction displayed for approval
Sync/pairing direction (CryptoMultiAccounts)	Sync QRs are unsigned — no device-to-host authentication
Vendor test suite	No binding/mismatch tests exist — the absence is by design

Why this is systemic rather than four bugs. The product's core guarantee — "the device screen is ground truth" — holds only for the request direction. For results and pairing, nothing binds what the user saw on the device to what the host broadcasts or stores. Every integrating wallet inherits the gap and must independently re-implement binding, which is impossible for the legacy-coin path (the SDK never even exposes the expected id alongside the result).

Severity calibration. Individual paths require host-side control of QR rendering (precondition: malicious/compromised host application — the industry's demonstrated loss vector). Reported HIGH (systemic) per conservative gating; the aggregate defeats the product's security proposition across every integrating wallet and is CRITICAL-candidate pending the vendor's threat-model position on malicious hosts (which their air-gapped value proposition necessarily implies).

Remediation. One architectural change closes the class: (1) mandatory session-id equality check in every parse path; (2) device-side signature/MAC over result payloads keyed to the request session; (3) diff returned rawTx against the unsigned transaction that was displayed; (4) signed sync payloads. Until shipped, integrators cannot fully mitigate: the SDK discards the data needed for binding on some paths.

4.2 [F13] Legacy-coin signing (LTC/BCH/DASH/DOGE): response binding entirely absent — substituted results silently accepted

Severity: HIGH **Status:** CONFIRMED — runtime PoC against published `@keystonehq/keystone-sdk` 0.12.3 **Location:** `src/chains/keystone/index.ts:112-127` (`parseSignResult`) **CWE:** CWE-345, CWE-347, CWE-602

Code. The request carries `signId` (the session id) into the protobuf payload. The response parser:

```
parseSignResult (ur: UR): KeystoneSignResultType {
  ...
  const value = base.data?.Content.value
  const requestId = value.signId          // ← read from response
  return { requestId, rawData: value.rawTx } // ← returned verbatim
}
```

// ← never compared to the request's id

The response's `signId` is echoed back as if it were the host's expected id. The signed `rawTx` is returned with no validation against the transaction that was displayed. Contrast: the Ethereum-family keyrings at least perform the comparison when the field is present — this protocol performs none, and the SDK offers integrators no way to perform it (the expected id is not plumbed through).

PoC reproduction (deterministic, no device required). 1. Host builds a request via `new Keystone().generateSignRequest({ chain: "LTC", requestId: "6ba7b810-9dad-41d1-80b4-00c04fd430c8", ... })`. 2. Attacker crafts a result payload: `protobuf SignTransactionResult { signId: "ffffffff-ffff-4fff-8fff-ffffffffffff", rawTx: "SIGNED-TX-SEND-ALL-T0-ATTACKER" }`, wrapped in the standard gzip/Base envelope, encoded as a UR QR. 3. Host scans (or renders) the forged QR and calls `parseSignResult`. 4. **Observed:** no error; returns `{ requestId: "ffffffff-...", rawData: "SIGNED-TX-SEND-ALL-T0-ATTACKER" }` — the mismatched session id and attacker transaction are accepted verbatim.

Attack narrative. Victim initiates a legitimate LTC send and approves it on the device. The host application (malicious dapp, or compromised frontend rendering its own QR — the Bybit/Safe pattern) substitutes a forged result QR. The victim scans it believing it is the device's output; the host broadcasts the attacker's pre-signed transaction. The user watched their device approve one transaction; the network settled another.

Impact. 100% of the wallet's balance in that asset, per session, with a valid on-chain signature and no broken UX to raise suspicion. Applicable to LTC, BCH, DASH, DOGE — four assets in one code path.

Artifacts: `findings_F13.json`, `run_poc_f13_v1.txt`, `poc/poc_f13_keystone_binding.js`, `poc/poc_f13_visual_chain.js` (generates scannable QR evidence images), `f13_visual_chain.json`.

4.3 [F7] Signature request binding is optional — substitution accepted when the binding field is omitted

Severity: HIGH (context: malicious/compromised host pairing flow) **Status:** CONFIRMED — runtime PoC against published `@keystonehq/base-eth-keyring` 0.15.1 **Location:** `packages/base-eth-keyring/src/BaseKeyring.ts:132`; same pattern replicated in `sol-keyring` (:59), `cosmos-keyring` (:61), `aptos-keyring` (:55) **CWE:** CWE-347

Code.

```
const requestIdBuffer = ethSignature.getRequestId();
if (requestIdBuffer) { // ← guard skipped when field absent
  const requestId = uuid.stringify(requestIdBuffer);
  if (requestId !== _requestId) { throw ... } // ← the only binding that exists
}
```

PoC reproduction. 1. Host requests a signature for message A (request id `6ba7b810-...`). 2. The "device response" delivered to the scanner carries a signature over attacker content **with the requestId field absent**. 3. **Observed:** `ACCEPTED – no binding check performed` (the keyring returned r/s/v components from the attacker response; the mismatch path was never reached). 4. Control: identical response *with* a mismatched, present requestId → correctly rejected. The guard exists but is skippable by field omission — precisely the difference a malicious pairing flow needs.

Impact. Same substitution outcome as 4.2 for the ETH/SOL/COSMOS/APTOS keyring integrations (including the MetaMask snap ecosystem). Older firmware responses and hostile pairing flows that omit the field sail through.

Remediation. Make the check mandatory: reject any response lacking the field. Fail closed.

Artifacts: `findings_F7.json`, `run_poc_f7_v1.txt`, `poc/poc_f7_substitution.js`.

4.4 [F8] Remote configuration overrides caller config and controls the blind-signing downgrade threshold

Severity: HIGH (CONFIRMED + privileged precondition) **Status:** CONFIRMED — three PoC stages against the live endpoint and published SDK; cache-poisoning variants tested and rejected

Locations: `src/sdk.ts:31-50` (fetch/merge), `chains/sui.ts:66-93`, `chains/iota.ts:66`, `chains/cardano.ts:142,169` (consumers); device-side `rust/apps/sui/src/lib.rs:128` (`sign_hash`) **CWE:** CWE-345, CWE-602

Code.

```
const CONFIG_URL = "https://keyst.one/statics/sdk/config.json";
static async create (config?) {
  const res = await fetch(CONFIG_URL)
  const onlineConfig = await res.json()
  return new KeystoneSDK({ ...config, ...onlineConfig }) // remote wins
}
```

The live object (verified): `{"sizeLimit":{"sui":2048,"ada":2048,"iota":2048}}`. It controls the threshold at which Sui/Iota/Cardano messages are **hashed client-side** and transmitted to the device as a bare 32-byte digest (`SuiSignHashRequest`), which the device signs via `sign_hash()` without any transaction content to display.

PoC chain (three stages, all observed): 1. **Remote-wins override.** Caller passes `{ sizeLimit: { sui: 4096 } }`; `KeystoneSDK.create()` yields `2048` — the live remote value silently overrides the integrator's choice. 2. **Downgrade trigger.** A 3,000-byte intent message (above the remote threshold) produces a `sui-sign-hash-request` UR; a 100-byte message produces a full `sui-sign-request`. 3. **Device-view decode.** UR-level decoding of the downgraded request proves the message hash is transmitted **without any intent message** — the device cannot display recipient, amount, or calls because they are not sent. Whatever digest the host presents, the device signs.

Attack narrative. Whoever controls that one configuration object (bucket/CI/insider — all externally reachable write paths were probed and closed; see §7) can lower the threshold to zero, converting **every Sui/Iota/Cardano approval on every integrating dapp into blind signing simultaneously** — no app update, no user-visible change. A drainer dapp then needs only one "claim your airdrop" approval to extract a user's entire portfolio on those chains. This is the Bybit/Safe event shape at the SDK layer.

Rejected variants (documented for completeness): CORS reflection poisoning (cache serves wildcard `ACA0:*` — no per-origin value to poison); direct write via distribution (GET/HEAD-only); bucket discovery (no references in 103 bundles, all repos, or DNS).

Remediation. Pin security-relevant limits locally; treat remote config as untrusted input (never a source of security thresholds); integrity-verify any remote configuration; make hash-only downgrades explicit on-device.

Artifacts: `findings_F8.json`, `run_poc_f8_v1_headers.txt`, `run_poc_f8_v5_deviceview.txt`, `poc/poc_f8_v3.js`, `poc/poc_f8_device_view.js`, `poc/poc_f8_v3_result.json`.

5. Findings — MEDIUM-HIGH / MEDIUM

5.1 [F11] Silent input truncation on the signing path — valid signatures over unintended data

Severity: MEDIUM-HIGH · **Status:** CONFIRMED (runtime PoC) `toBuffer` = `Buffer.from(str, "hex")` silently drops invalid hex characters. A host passing `0x`-prefixed or malformed hex (a common integration bug) obtains a **valid signature over truncated — often empty — data** with no error. PoC: a message of `"zz"` + 3,000 bytes produced a normal `sui-sign-request` over the truncated buffer; no exception raised. Remediation: strict hex validation; fail closed on empty results; assert encoded length equals declared message length. Artifacts: `findings_F11.json`, `poc/poc_f8_v3.js` (case 2b).

5.2 [F9] Tron display parser: single-contract parsing and fabricated display fields

Severity: MEDIUM-HIGH · **Status:** CONFIRMED (code) `raw2tx` (`chains/tron/transfer.ts:37,94-95`) parses only `rawData.contract[0]` — additional contracts in a multi-contract Tron transaction are signed but never displayed. For `TriggerSmartContract`, displayed recipient/amount are extracted from fixed calldata offsets assuming a TRC-20 layout regardless of the actual function selector — non-token calls display fabricated recipient and value. Note the asymmetry: the *device* firmware Tron parser (Rust) is selector-gated and fail-closed; the *host* SDK is not. Trust divergence is the finding.

5.3 [F4] Missing Content-Security-Policy and Permissions-Policy

Severity: MEDIUM · **Status:** CONFIRMED (live headers) For a crypto-brand storefront carrying analytics/ad tags (Mixpanel, Meta, affiliate networks, chat widget), absence of CSP means a single injected script or compromised tag leads directly to skimming/hooks on shop pages. Shopify-hosted checkout limits exposure to the storefront session. Remediation: deploy CSP (report-only → enforce), Permissions-Policy.

5.4 [F1/F2] Production client falls back to the staging API; staging exposed

Severity: MEDIUM (each) · **Status:** CONFIRMED (live) Production bundle resolves the API host via `{staging, production, development}[window.ENV || "staging"]` — **any unset/unexpected environment value routes shopper cart and discount traffic to `api-staging.keyst.one`**. Staging is internet-reachable, mirrors production's secret-protected surface, and exhibits drift (`/v2/web/shop_products/` 404 on staging, live on production). Remediation: default to production; restrict staging ingress; alert on staging traffic.

5.5 [F12] Firmware Tron amount display uses an f64 pipeline — display collision at high values

Severity: MEDIUM · **Status:** CONFIRMED (mechanism PoC; exploitation not practical) `format_amount` (`rust/apps/tron/src/transaction/wrapped_tron.rs`) parses raw amounts as f64 and divides by a float divider. PoC (exact firmware logic recompiled): raw values `18446744073709551615` and `18446744073709550591` (1,024 raw units apart) render **identically** as `"18446744073709.550781"`

`USD"` ; U256-scale values display with corrupted low-order digits. Honest transfers never reach the collision region; the ETH app's integer math (exact BigInt) demonstrates the correct pattern available in-codebase. Remediation: fixed-point integer formatting.

6. Findings — LOW

ID	Finding	Status	Detail
F3	Verbose backend error disclosure (CWE-209)	CONFIRMED	<code>/v2/web/shop_cart/</code> returns upstream internals: <code>Failed to fetch shopping cart ('shopify', 'response error')</code> and parameter-level detail <code>(API params 'id' is missing or null)</code> — confirms architecture and parameter names at zero cost to attackers. Normalize errors.
F5	WAF User-Agent filter trivially bypassed	CONFIRMED	Scanner/empty User-Agents → 403; browser UA → 200. Defense-in-depth theater; bypass documented in bypass matrix.
F6	HSTS gaps	CONFIRMED	No <code>includeSubDomains</code> , no <code>preload</code> — subdomain TLS-stripping risk remains for the surrounding host ecosystem.
F10/ F14	Weak path parsing; malformed sync-QR crash	CONFIRMED (code)	<code>parsePath</code> accepts <code>Number()</code> coercions (<code>"</code> → 0, <code>1e2</code> → 100) — silent mis-derivation risk; <code>parseMultiAccounts</code> indexes <code>origin.getComponents()[1]</code> unguarded — host-side <code>TypeError</code> on malformed sync QR.

7. PoC Verification Discipline

Every CONFIRMED finding passed the three-variant gate; every failed hypothesis is recorded (internal state) rather than silently dropped. Summary of the gate's outcomes:

Finding	Variants attempted	Result
F13 substitution	Forged result accepted; dist-vs-repo identity check; QR evidence generation	CONFIRMED
F7 substitution	Response-without-id accepted; mismatched-id rejected (control); 4 keyrings source-verified	CONFIRMED
F8 downgrade	Live override observed; downgrade trigger observed; device-view decode observed; cache-poisoning REJECTED; write-surface REJECTED (4 vectors)	CONFIRMED (HIGH)
F11 truncation	Corrupted-hex accepted silently; normal input unaffected (control); dist identity check	CONFIRMED
Discount enumeration via API	Missing/empty/null/garbage Turnstile tokens — all rejected server-side; rate limit active	REJECTED
Cart IDOR	Shopify cart ids unguessable; generic upstream errors; no enumeration oracle	REJECTED
Config critical escalation	Distribution PUT 403; bucket undiscoverable; listing empty; cache poisoning rejected	REJECTED (severity capped)
Seed-entropy weakness (Coldcard-class)	Current HEAD uses HKDF-chained 3-source RNG, error-checked, <code>memset_s</code>	DOES NOT REPRODUCE

8. Hardware (device firmware) Assessment — No Critical Found

The device-side money-display chain was reviewed end-to-end and found **fail-closed at every layer**: ETH/ERC-20 (exact-length calldata, integer BigUint amounts), EIP-712 (raw-hex fallback, SafeTx verification), Tron (selector-gated, raw-units fallback for unknown tokens, unknown contracts rejected), Solana (undecodable transactions explicitly declared), batch/swap rules (strict shape constraints with test coverage), BTC/PSBT (output addresses from the actual scriptPubKey, prevout tamper check, **change-output detection bound to the device's real master fingerprint**, multisig+taproot rejected), firmware update (dual signature verification at init and completion, hash-content binding, failure deletes the staged file, update denied while wallet unlocked, key rotation under a separate root), and secrets (none hardcoded in firmware, SDKs, or CI). The Keylabs 2023 device audit's HIGH finding (tamper response) was reported fixed and retested; the Coldcard-class seed-entropy failure mode does not reproduce at current HEAD. The single firmware defect is the f64 display pipeline (5.5, MEDIUM). **The device signs what it displays and displays what it signs; the vulnerability class identified in this report lives between the camera and the broadcast — in the host-side SDK.**

9. Threat Intelligence Cross-Check

- **Bybit/Safe (Feb 2025, \$1.5B)**: blind signing via compromised host frontend — the exact loss class confirmed reachable here through 4.2/4.3/4.4. The industry's costliest hardware-wallet failure occurred *in the layer this assessment finds unauthenticated*.
- **Coldcard seed heist (Aug 2026, ~\$130M)**: predictable seed generation. Keystone's equivalent path was re-verified at current HEAD and does not reproduce.

- **Keylabs audit (Nov 2023, device-only scope):** complements this assessment; all vendor audits to date cover the device — **the host-side SDK acceptance layer was previously unaudited** and contains the systemic finding.
- **Vendor bug bounty:** active program; findings 4.1–4.4 are submission-ready with reproducible PoCs.

10. Remediation Priorities

1. **Make result binding mandatory everywhere (closes F13/F7/F15):** reject any scan result whose session id does not equal the request's; reject responses lacking the id; plumb the expected id through every parse API. Highest urgency.
2. **Validate returned rawTx/signatures against the displayed transaction** (diff inputs/outputs) — device-side signature over result payloads keyed to the session is the durable fix.
3. **Remove security-relevant fields from remote configuration** (closes F8): pin `sizeLimit` locally; never let remote values weaken thresholds; fail closed on config fetch errors; make hash-only downgrades explicit on-device.
4. **Strict input validation on the signing path** (closes F11): reject invalid/ `0x`-prefixed hex; fail closed on empty encodings.
5. Host-side display parity with firmware parsers (closes F9): parse all contracts; fail closed on unknown selectors.
6. Web hardening: CSP/Permissions-Policy (F4), production-default API routing + staging lockdown (F1/F2), normalized API errors (F3), HSTS scope (F6).
7. Firmware: replace the Tron f64 display pipeline with fixed-point integer math (F12); add binding tests to the SDK test suites (the absence of such tests is how this class shipped).

11. Fund-Loss Mechanics (aggregate)

Path	Per-victim loss	Ecosystem ceiling	Preconditions
4.2/4.3 substitution	100% of wallet balance per signing session	Every integrator's userbase (drainer-kit scale)	Malicious/compromised host app; one QR swap
4.4 blind-signing downgrade	100% of portfolio per approval	All Sui/Iota/ADA users ecosystem-wide (Bybit event shape)	Config write access (privileged)
5.1 truncation	Valid signature over wrong data	All integrations with common input bugs	Buggy/malicious host

The class defeats the hardware wallet's core promise — "nothing moves without the device screen showing it" — because that promise holds only for the request direction: the device shows one transaction, the chain settles another, and nothing between camera and broadcast verifies the match.

Appendix A — Evidence Index

- Findings: `findings_F7.json`, `findings_F8.json`, `findings_F11.json`, `findings_F12.json`, `findings_F13.json`, `findings_F15.json`

- PoC sources: `poc/poc_f7_substitution.js`, `poc/poc_f8_v3.js`, `poc/poc_f8_device_view.js`, `poc/poc_f13_keystone_binding.js`, `poc/poc_f13_visual_chain.js`, `poc/poc_tron_float.rs`
- PoC outputs: `run_poc_f7_v1.txt`, `run_poc_f8_v1_headers.txt`, `run_poc_f8_v5_deviceview.txt`, `run_poc_f13_v1.txt`, `poc/poc_f8_v3_result.json`, `poc/f13_visual_chain.json`
- QR evidence: `poc/qr_1_device_request.png`, `poc/qr_2_attacker_result.png`
- Reconnaissance: `target_profile.json`, `phaseA1_stack.json`, `bypass_matrix.json`, `run_whatweb_S1.txt`, `run_wafw00f_S1.txt`, `run_sslyze_S1.txt`, `run_nmap_full_S1.txt`, `run_nikto_S1.txt.txt`, `run_dnsrecon_S1.txt`, `run_crtsh_S1.txt`, `run_ffuf_dirs_S2.json`, `services_enum_summary.txt`
- Endpoint inventory: `structure/endpoints_master.txt` (6,270 entries), `structure/api_routes_all.txt` (13 API routes), `structure/katana_*.txt`
- Client bundles: `js/` (103 bundles), `js_leaks` -class scans; source snapshots: `repos/` (keystone-sdk-web, keystone-sdk-base, keystone3-firmware, Smart-Contract-Metadata-Registry, Keystone-developer-hub)

All testing was conducted with respect for production guardrails; no customer data was accessed; no financial transactions were created; no availability impact occurred.